Estadística con Python y CRUD

Un recorrido completo desde el análisis descriptivo hasta las bases de datos, pasando por simulaciones, inferencia bayesiana y contrastes de hipótesis. Todo con código Python real y listo para aplicar.

MÓDULO 5.3

PYTHON · PANDAS · SCIPY · SQLALCHEMY



Estadística Descriptiva con Pandas

Pandas ofrece un conjunto de herramientas potente para obtener un primer vistazo cuantitativo de cualquier conjunto de datos de forma inmediata.

Métodos clave

- **df.describe()** — media, std, cuartiles
- **df.mean() / df.median()** — tendencia central
- **df.corr()** — correlaciones entre variables
- **df.value_counts()** — frecuencias absolutas

¿Por qué es el primer paso?

Antes de modelar o inferir, necesitas entender la forma, el centro y la dispersión de tus datos. La estadística descriptiva detecta valores atípicos, distribuciones sesgadas y variables constantes que podrían distorsionar cualquier análisis posterior.



Distribuciones de Probabilidad con SciPy

scipy.stats implementa más de 100 distribuciones continuas y discretas, desde la Normal hasta la Poisson, con una interfaz unificada y consistente.



Normal y t de Student

Para modelar fenómenos continuos y realizar inferencias con muestras pequeñas. **norm.pdf()**, **t.cdf()**.



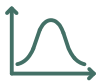
Binomial y Poisson

Distribuciones discretas para contar eventos. Esenciales en experimentos A/B y análisis de frecuencias.



Ajuste de distribución

Con **scipy.stats.fit()** puedes encontrar automáticamente la distribución que mejor se ajusta a tus datos empíricos.



Generación de muestras

Usa **.rvs(size=n)** para generar observaciones aleatorias desde cualquier distribución parametrizada.

Simulación Monte Carlo

Monte Carlo convierte problemas analíticamente complejos en experimentos computacionales: se generan miles de escenarios aleatorios y se observa el comportamiento agregado del sistema.

1 · Definir el modelo

Identificar variables inciertas y sus distribuciones de probabilidad.

2 · Muestrear

Generar N realizaciones con **np.random** o **scipy.stats.rvs()**.

3 · Simular

Calcular el resultado del modelo para cada muestra generada.

4 · Agregar

Obtener media, percentiles e intervalos de confianza de los resultados.

Cuanto mayor sea N (número de simulaciones), más precisa será la estimación. Valores típicos: de 10.000 a 1.000.000 iteraciones.



Bayes en Código Python

El enfoque bayesiano actualiza nuestras creencias previas (*prior*) a la luz de nuevos datos para obtener una creencia actualizada (*posterior*). Python permite implementarlo de forma explícita y transparente.

Teorema de Bayes

$$P(H|D) = \frac{P(D|H) \cdot P(H)}{P(D)}$$

Donde **P(H)** es el prior, **P(D|H)** es la verosimilitud y **P(H|D)** es el posterior que buscamos calcular.

Implementación práctica

- **PyMC** — modelado probabilístico completo
- **NumPy** — Bayes discreto manual
- **scipy.stats** — distribuciones conjugadas

Ejemplo clásico

Test médico con tasa de falsos positivos: dado un resultado positivo, ¿cuál es la probabilidad real de estar enfermo? Bayes responde combinando la prevalencia de la enfermedad con la precisión del test.

Contraste de Hipótesis e Intervalos de Confianza

El contraste de hipótesis permite decidir, con base estadística, si una diferencia observada en los datos es real o producto del azar. El **t-test** es el punto de entrada más utilizado.

t-test de una muestra

Contrasta si la media de una muestra difiere de un valor de referencia teórico.

scipy.stats.ttest_1samp()

t-test de dos muestras

Compara las medias de dos grupos independientes. Fundamental en experimentos A/B.

scipy.stats.ttest_ind()

p-valor e interpretación

Si $p < 0.05$, rechazamos H_0. El p-valor no mide la magnitud del efecto, solo la evidencia contra la hipótesis nula.

Intervalos de confianza

Rango de valores plausibles para el parámetro. Más informativos que el p-valor solo.

scipy.stats.t.interval()



CRUD desde Python

Conectar Python con bases de datos relacionales y NoSQL es una habilidad imprescindible para cualquier analista de datos moderno.

¿Qué es CRUD?

Create · **R**ead · **U**pdate · **D**elete — las cuatro operaciones fundamentales sobre cualquier base de datos, ya sea SQL o NoSQL.

sqlite3

Módulo estándar de Python para bases de datos relacionales locales. Sin instalación adicional, ideal para prototipado y aprendizaje.

SQLAlchemy

ORM y toolkit SQL de referencia en Python. Abstrae el dialecto SQL y permite trabajar con PostgreSQL, MySQL o SQLite sin cambiar el código.

pymongo

Driver oficial de MongoDB para Python. Permite operaciones CRUD sobre colecciones de documentos JSON con una API intuitiva.

sqlite3: La Base de Datos en un Fichero

El módulo **sqlite3** viene incluido en la librería estándar de Python y permite crear y gestionar bases de datos relacionales sin necesidad de instalar ningún servidor externo.

```
import sqlite3

# Conectar (crea el fichero si no existe)
conn = sqlite3.connect("datos.db")
cursor = conn.cursor()

# CREATE TABLE
cursor.execute("""
 CREATE TABLE IF NOT EXISTS ventas (
 id INTEGER PRIMARY KEY,
 producto TEXT,
 importe REAL
 )
""")

# INSERT (Create)
cursor.execute("INSERT INTO ventas VALUES (1, 'Laptop', 999.99)")

# SELECT (Read)
for row in cursor.execute("SELECT * FROM ventas"):
 print(row)

conn.commit()
conn.close()
```

SQLAlchemy y el Puente `to_sql`

SQLAlchemy es la biblioteca ORM de referencia en Python. Combinada con pandas, el método **to_sql()** permite volcar un DataFrame completo a una tabla SQL en una sola línea de código.

Con SQLAlchemy ORM

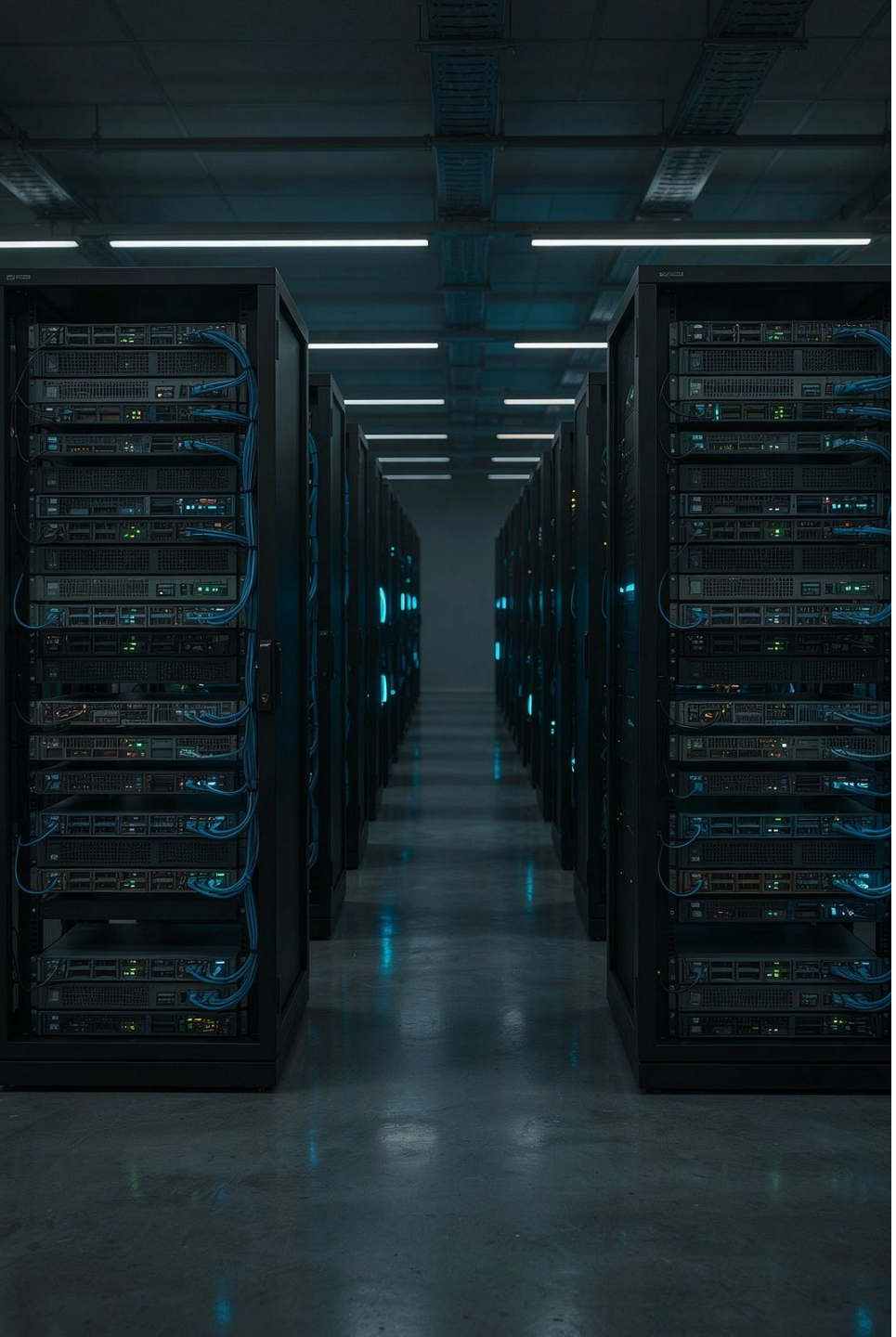
Define modelos como clases Python. El motor gestiona la traducción a SQL automáticamente, soportando múltiples motores: SQLite, PostgreSQL, MySQL.

- **create_engine()** — conexión al motor
- **Session()** — contexto transaccional
- **Base.metadata.create_all()** — crea tablas

DataFrame → SQL con pandas

El método **df.to_sql()** es la forma más rápida de persistir resultados de análisis directamente en una base de datos, sin escribir SQL manualmente.

```
df.to_sql(
 "resultados",
 con=engine,
 if_exists="replace",
 index=False
)
```



pymongo: CRUD en MongoDB

MongoDB almacena documentos JSON en lugar de filas tabulares. **pymongo** expone una API directa y expresiva para trabajar con colecciones, ideal para datos semiestructurados o con esquema variable.

Conectar e insertar

```
client = MongoClient("mongodb://localhost:27017")
db.ventas.insert_one({"producto": "Tablet", "importe": 349})
```

Consultar (Read)

```
db.ventas.find({"importe": {"$gt": 200}})
```

Usa operadores como **$gt**, **$lt**, **$in** y **$regex** para filtros potentes.

Actualizar y eliminar

```
db.ventas.update_one(filtro, {"$set": {"importe": 399}})
db.ventas.delete_many({"importe": {"$lt": 10}})
```

S05 · EN EL REPOSITORIO DEL CURSO

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO **S05_04_estadistica_descriptiva_ic** — 35 min
Descriptiva, Monte Carlo e intervalos de confianza
sprints/S05/notebooks/S05_04_estadistica_descriptiva_ic.ipynb

CUADERNO **S05_05_contrastes_causalidad** — 35 min
Contrastes, causalidad, statsmodels y Bayes
sprints/S05/notebooks/S05_05_contrastes_causalidad.ipynb

CUADERNO **S05_06_crud_python** — 30 min
CRUD con sqlite3 y mongomock (es también el paso 1 del reto: cargar los datos en una base)
sprints/S05/notebooks/S05_06_crud_python.ipynb

RETO **Reto del sprint** — 45 min
Cuaderno S05_07: mirar los datos → regresión desde cero → comparar con una librería → medir la seguridad del…
sprints/S05/guia/Reto_Sprint_05.md